TECHNICAL ARCHITECTURE & ALGORITHMS

Investment Committee

Model integration, the consensus algorithm, risk sizing, and the execution pipeline — with the exact thresholds and formulas in use.

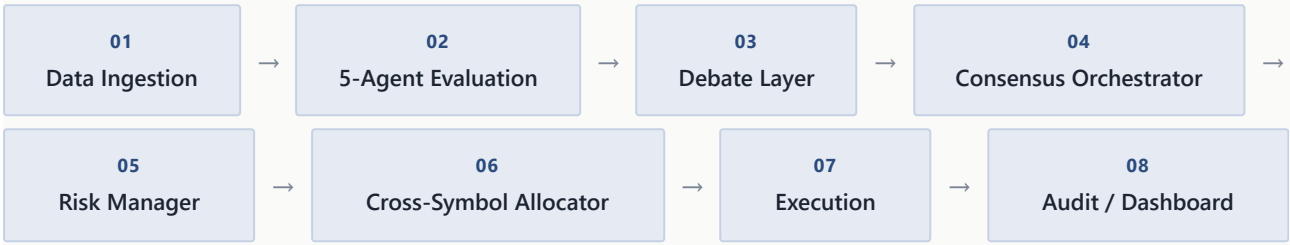
DOCUMENT
Technical Architecture Report

DATE
8 July 2026

COMPANION DOCUMENT
Business Briefing

1 System Overview

Each five-minute cycle pushes every watchlist symbol through the same nine-stage pipeline: data → five independent agent evaluations → a debate pass → consensus scoring → risk sizing → cross-symbol capital allocation → simulated execution → persistence → the audit/dashboard layer.



Implemented in `backend/committee/orchestration/{cycle,loop}.py`, run once per symbol per five-minute cycle across the full watchlist.

2 Data Layer

OHLCV bars are pulled via **yfinance** (`market_data/prices.py`) at 5-minute resolution for live decisioning. Each fetch is merged into a growing per-symbol cache — `data/historical/{SYMBOL}_{interval}.csv` — with duplicate timestamps resolved in favor of the newer pull, so the effective history accumulates past yfinance's ~60-day intraday window limit over time. A separate, coarser cache (15-minute bars, 60-day window) feeds offline model training via `scripts/train_forecasting_model.py`. If a live fetch fails, the pipeline falls back to the last cached bars rather than erroring the cycle. News comes from public RSS (Moneycontrol, Economic Times) via `market_data/news.py`.

3 The Five Agents

A deliberately heterogeneous committee: one rule-based, two LLM-backed, one adversarial LLM, one gradient-boosted tree model — chosen so the agents fail in different ways rather than sharing blind spots.

Technical

RULE-BASED

Weighted sum over RSI, EMA-cross, MACD, and momentum signals: weights `0.3 / 0.3 / 0.2 / 0.2`, clipped to `[-1, 1]`. Below `|score| < 0.15` (`WAIT_BAND`) the agent abstains. Pure pandas/numpy, no external calls. `agents/technical.py`

News & Sentiment

LLM — GEMINI

Reads recent headlines for the symbol and scores sentiment/direction. Routed via `llm/router.py` to Gemini (`gemini-1.5-flash`). Context relevance is boosted 1.5× on earnings days. `agents/news_sentiment.py`

Macro

LLM — OPENAI

Weighs rates, policy, and sector-wide conditions. Routed to OpenAI (`gpt-5-mini`). Context relevance boosted 1.8× on RBI policy days. `agents/macro.py`

Forecasting

LIGHTGBM

Multiclass classifier (bearish / neutral / bullish) trained offline on lagged returns (1,2,3,5,10 bars), RSI, MACD histogram, EMA20–50 diff, rolling volatility, and volume change; predicts 3 bars ahead. Zero language reasoning — pure statistical pattern-matching, deliberately independent from the LLM agents. `agents/forecasting.py` , trained by `scripts/train_forecasting_model.py`

Contrarian

LLM — ANTHROPIC

Runs on Claude Haiku (`claude-haiku-4-5`) — a different provider on purpose, so its disagreement reflects a genuinely different model rather than one model role-playing four personas. Issues its own vote plus an explicit challenge and risk observations, consumed by the Debate Layer below. `debate/contrarian.py`

AGENT	BASE EXPERTISE	PROVIDER
Technical	1.2	—
News & Sentiment	1.0	Gemini (gemini-1.5-flash)
Macro	0.8	OpenAI (gpt-5-mini)
Forecasting	1.0	LightGBM (local)

Contrarian	0.6	Anthropic (claude-haiku-4-5)

Source: `config.py::BASE_EXPERTISE`, `AGENT_PROVIDER_MAP`.

4 Volatility-Scaled Forecasting Labels

Training labels for the Forecasting agent scale to each stock's own volatility rather than a fixed return threshold, so a calm stock's noise and a volatile stock's real moves aren't pooled under one cutoff:

```
# agents/forecasting.py :: build_labels
horizon_vol = rolling_std(returns, window=10) * sqrt(lookahead=3)
deadzone = max(0.5 * horizon_vol, 0.0005) # floor
label = +1 if forward_return_3bar > deadzone
-1 if forward_return_3bar < -deadzone
0 otherwise (neutral)
```

Constants: `FORECAST_VOLATILITY_WINDOW=10`, `FORECAST_LOOKAHEAD_BARS=3`, `FORECAST_DEADZONE_VOL_MULTIPLIER=0.5`, `FORECAST_DEADZONE_MIN_RETURN=0.0005`.

5 Debate Layer

Before consensus is computed, any agent whose vote disagrees with the Contrarian has its confidence damped, scaled by the Contrarian's own confidence — so a low-confidence dissent barely moves anything, while a highly confident one can roughly halve it:

```
# debate/engine.py
revised_confidence = confidence * (1 - contrarian_confidence * REVISION_DAMPING_FACTOR)
# REVISION_DAMPING_FACTOR = 0.5
```

6 Consensus Algorithm

Each agent emits a signed vote — positive confidence for BUY, negative for SELL, zero for WAIT. Votes are combined into a single **Directional Confidence Score (DCS)**, weighted by each agent's influence:

```
# trust/scoring.py + consensus/orchestrator.py
influence_i = confidence_i * trust_i * context_relevance_i
norm_influence_i = influence_i / sum(influence_j for all agents j)
DCS = sum(norm_influence_i * signed_vote_i) # in [-1, 1]

if |DCS| < DECISION_THRESHOLD_WAIT (0.15): WAIT
else: BUY/SELL, allocation = min(|DCS| * LEVERAGE, LEVERAGE)
```

Trust score is a Laplace-smoothed historical hit rate, updated every cycle by checking whether the agent's prior call matched the next 2-bar price direction:

```
# persistence/repository.py :: update_trust_score
trust = (correct_calls + TRUST_PRIOR * TRUST_SMOOTHING) / (total_calls + TRUST_SMOOTHING)
# TRUST_PRIOR = 0.5, TRUST_SMOOTHING = 2.0 — starts neutral, converges with evidence
```

Context relevance = `BASE_EXPERTISE` (Section 3 table) × a situational multiplier keyed off a per-cycle context flag — `earnings_day` boosts News & Sentiment 1.5×, `rbi_policy_day` boosts Macro 1.8×, otherwise all multipliers are 1.0.

7 Risk Management Layer

An independent GARCH(1,1)-estimated annualized volatility (`risk/volatility.py`) gates every consensus decision before it can become an order:

CONDITION	ACTION	CONSTANT
Annualized vol > 100%	**Reject trade outright**	EXTREME_VOLATILITY_ANNUALIZED = 1.0
Annualized vol > 45%	**Trim allocation 50%**	HIGH_VOLATILITY_ANNUALIZED = 0.45 / VOLATILITY_TRIM_FACTOR = 0.5
Any approved trade	**Cap at 100% of buying power**	MAX_POSITION_ALLOCATION = 1.0

GARCH volatility is annualized from 5-minute bars using `INTRADAY_BARS_PER_DAY=75` and `TRADING_DAYS_PER_YEAR=252` . `risk/manager.py`

8 Cross-Symbol Capital Allocator

Per-symbol risk sizing is computed independently and can, in aggregate, request more capital than is available. The allocator (`orchestration/allocator.py`, run after per-symbol risk evaluation and before execution) reconciles this confidence-weighted rather than first-come-first-served:

```
# orchestration/allocator.py
headroom = BUYING_POWER - current_gross_exposure
for symbols increasing exposure, if sum(incremental_needs) > headroom:
  share_i = confidence_i / sum(confidence_j for needy symbols j)
  final_allocation_i = (current_gross_i + headroom * share_i) / BUYING_POWER
  verdict_i = REDUCE
# symbols reducing exposure are never throttled
```

9 Execution & Cost Model

Execution (`execution/portfolio.py`) is fully simulated — no broker API is present anywhere in the codebase. It trades only the delta versus the current in-memory position, against a virtual portfolio seeded at `CAPITAL = ₹10,000 × LEVERAGE = 2 → BUYING_POWER = ₹20,000` . Every fill applies the NSE intraday retail cost schedule (`execution/cost_model.py`):

COST COMPONENT	RATE	APPLIES TO
Brokerage	0.03%, capped at ₹20	Both sides
STT	0.025%	Sell side only
Exchange transaction charge	0.00297%	Both sides
SEBI charge	₹10 / crore	Both sides
Stamp duty	0.003%	Buy side only
GST	18%	On brokerage + exchange charge
Slippage	0.02%–0.05% (midpoint applied)	Both sides — deterministic approximation, not a market-impact model

There is also a **Replay Mode** (`replay/player.py`) that streams cached historical bars through the identical pipeline at accelerated speed — a lightweight backtester reusing the live code path — plus a **vectorbt SMA-crossover baseline** (`baseline/vectorbt_baseline.py` , `scripts/compare_baseline.py`) used purely as a performance benchmark, not part of the live decision path.

10 Audit & API Layer

P&L is computed mark-to-market rather than by summing realized trade cash flows, so it correctly includes unrealized P&L on open positions (`audit/report.py::summarize_pnl`):

```
# audit/report.py
net_pnl = portfolio_value (mark-to-market) - starting_capital
gross_pnl = net_pnl + total_costs # adds back all costs actually charged
growth_pct = (net_pnl / starting_capital) * 100
```

Backend is **FastAPI** (`api/main.py`), CORS-scoped to the local Vite dev origin only:

ENDPOINT	METHOD	PURPOSE
/health	GET	Liveness check
/cycle/{symbol}	POST	Run one committee cycle for a symbol
/watchlist/run	POST	Run one cycle across the full watchlist
/decisions	GET	Full decision log — every agent vote, debate, consensus, risk verdict
/trades	GET	Executed trade history
/portfolio	GET	Current cash and open positions
/portfolio/curve	GET	Portfolio value over time (for the chart)
/report	GET	Gross/costs/net P&L summary + per-symbol cost breakdown

11 Frontend

Vite + TypeScript, vanilla DOM (no framework) — `frontend/src/main.ts` . Renders stat tiles (cash, gross/net P&L, growth %, costs, open positions, trade count), a portfolio value line chart, a decisions table with a click-through detail panel showing every agent's vote and reasoning plus the Contrarian's challenge, and a full trade log.

12 Tech Stack

LAYER	LIBRARIES
API	`FastAPI, uvicorn, pydantic-settings`
Data	`SQLAlchemy 2.0 (SQLite), pandas, numpy`
ML	`scikit-learn, lightgbm, arch (GARCH), vectorbt (baseline)`
Market data	`yfinance, feedparser`
LLM providers	`google-generativeai, openai, anthropic`
Frontend	`Vite, TypeScript`

13 Known Limitations

- **Single data vendor, no redundancy** — yfinance only, with a last-known-cache fallback but no secondary provider or failover.
- **Three of five agents depend on external LLM APIs** (Gemini, OpenAI, Anthropic) — introduces per-cycle latency, cost, and third-party availability risk into the decision path.
- **No automated stop-loss / max-drawdown kill-switch** — risk controls are volatility-based sizing/rejection plus a forced end-of-day square-off; there is no independent circuit breaker on cumulative losses.
- **No live-vs-backtest divergence tracking** beyond the vectorbt SMA baseline comparison — no formal live/replay parity monitoring yet.
- **Per-symbol cost attribution is diagnostic only** — shared cash across symbols makes fully rigorous per-symbol P&L attribution nontrivial (`audit/report.py::cost_breakdown_by_symbol`).